Premiers pas sur Python

Prise en main

1 | EXERCICE

Ouvrir le logiciel SPYDER.

Q1. Obtenir les valeurs approchées de
   a. 2^8, 2^{10}, 2^{16}.
   b. $\sqrt{3}$, $\sqrt[4]{5}$, $\sqrt[3]{5}$.

Q2. Le quotient dans la division euclidienne de 11^{10} par 9^{12} ; de ces deux nombres, lequel est le plus grand ?

Q3. Le reste dans la division euclidienne de 3^7 par 5.

Q4. Quelle expression permet d'obtenir le chiffre des 100 millions de 2^{100} ?

✎ CORRECTION DE L'EXERCICE 1

Q1. a.

console ×

```
In [1]: 2**8, 2**10, 2**16
Out[1]: (256, 1024, 65536)
```

b.

console ×

```
In [2]: 3**0.5, 5**0.25, 5**(1/3)
Out[2]: (1.7320508075688772, 1.4953487812212205, 1.7099759466766968)
```

Q2.

console ×

```
In [3]: 11**10 // 9**12
Out[3]: 0
```

Le quotient étant nul, 9^{12} est donc plus grand que 11^{10} (ce que confirme le calcul direct des deux puissances : $11^{10} = 25\,937\,424\,601 < 282\,429\,536\,481 = 9^{12}$).

Q3.

console ×

```
In [4]: 3**7 % 5
Out[4]: 2
```

Q4. Le chiffre des cent millions de 2^{100} est celui de position 10^8, qu'on isole en divisant 2^{100} par 10^8 (division entière) puis en prenant le reste modulo 10 :

console ×

```
In [5]: (2**100 // 10**8) % 10
Out[5]: 7
```

2 | EXERCICE
Fonctions hyperboliques

Implémenter deux fonctions Python `cosh` et `sinh` qui calcule le cosinus et le sinus hyperbolique d'un nombre réel.

✎ CORRECTION DE L'EXERCICE 2

hyperboliques.py ×

```python
from math import exp

def cosh(x):
    return (exp(x) + exp(-x)) / 2

def sinh(x):
    return (exp(x) - exp(-x)) / 2
```

console ×

```
In [1]: cosh(0), sinh(0)
Out[1]: (1.0, 0.0)

In [2]: cosh(1), sinh(1)
Out[2]: (1.5430806348152437, 1.1752011936438014)
```

matplotlib.pyplot

3 | EXERCICE

Q1. Dans l'éditeur saisir le code des deux fonctions suivantes, sans les commentaires, qui permettent, pour la première de tracer le graphe d'une fonction sur un intervalle :

graphe.py

```
1  from numpy import linspace       # Import de la fonction linspace
2  import matplotlib.pyplot as plt  # Import du module de trace
3
4  def graphe(f,a,b):     # Graphe de f au dessus de [a,b]
5      plt.figure(1)
6      X = linspace(a,b,100)   # abscisses
7      Y = [f(x) for x in X]   # ordonnees
8      plt.plot(X,Y)          # Trace
9      plt.axhline(color='black')
10     plt.axvline(color='black')
11     plt.show()
```

Q2. Les appeler pour tracer le graphe de `cosh` et `sinh` au dessus de l'intervalle $[-2\,;2]$.

Q3. Modifier le code des fonctions `cosh` et `sinh` de l'exercice 2 en utilisant `print` au lieu de `return`, puis retenter le tracé du (2). Que se passe-t-il ?

✎ Correction de l'exercice 3

Q1. Il suffit de saisir le code proposé dans l'énoncé (sans les commentaires), en respectant scrupuleusement l'indentation des blocs.

Q2. On obtient les deux graphes en appelant `graphe` avec `cosh` et `sinh` en argument :

console

```
In [1]: graphe(cosh, -2, 2)

In [2]: graphe(sinh, -2, 2)
```

Q3. En remplaçant `return` par `print` :

coshsinhprint.py

```
1  def cosh(x):
2      print((exp(x) + exp(-x)) / 2)
3
4  def sinh(x):
5      print((exp(x) - exp(-x)) / 2)
```

l'appel `graphe(cosh,-2,2)` affiche alors dans la console les 100 valeurs calculées par `linspace`, une par une – mais la fenêtre graphique s'ouvre avec un tracé **vide** : en l'absence de `return`, chaque appel `f(x)` renvoie `None` par défaut, si bien que la liste `Y` ne contient que des `None`, que `plt.plot` est bien incapable de représenter.

Structure de test

4 Exercice — Second degré

Écrire une fonction `trinome(a,b,c)` prenant en paramètre 3 entiers ou flottants $a \neq 0$, b, c, et qui renvoie les racines du trinôme $aX^2 + bX + c$.

✎ Correction de l'exercice 4

trinome.py

```
1  from math import sqrt
2
3  def trinome(a,b,c):
4      delta = b**2 - 4*a*c
5      if delta > 0:
6          x1 = (-b - sqrt(delta)) / (2*a)
7          x2 = (-b + sqrt(delta)) / (2*a)
8          return x1, x2
9      elif delta == 0:
10         x0 = -b / (2*a)
11         return x0
12     else:
13         partie_reelle = -b / (2*a)
14         partie_imaginaire = sqrt(-delta) / (2*a)
15         x1 = partie_reelle - partie_imaginaire*1j
16         x2 = partie_reelle + partie_imaginaire*1j
17         return x1, x2
```

console

```
In [1]: trinome(1,-3,2)     # delta > 0
Out[1]: (1.0, 2.0)

In [2]: trinome(1,-2,1)     # delta = 0
Out[2]: 1.0

In [3]: trinome(1,0,1)      # delta < 0
Out[3]: (-1j, 1j)
```

5 Exercice — Test de parité

Écrire une fonction `estpair(N)` qui teste la parité du nombre entier `N`. Il faudra utiliser l'opérateur `%`, qui retourne le reste de la division euclidienne.

✎ Correction de l'exercice 5

Un entier N est pair si et seulement si le reste de sa division euclidienne par 2 est nul :

estpair.py

```
def estpair(N):
    return N % 2 == 0
```

console

```
In [1]: estpair(4), estpair(7),
   estpair(0), estpair(-3)
Out[1]: (True, False, True, False)
```

6 Exercice
Rupture d'une corde

Une masse m est attachée à l'extrémité d'une corde de longueur $r = 3$ m. La masse ne peut tourner dans le plan horizontal qu'aux vitesses de 1, 10, 20 et 40 m.s^{-1}. La corde peut supporter une tension maximale $T = 60$ N avant de rompre.

Écrire une fonction `rupture(m)` qui a pour paramètre d'entrée la valeur de la masse m en kg et qui retourne la plus grande vitesse à laquelle la masse peut tourner sans que la corde ne se rompe.

Indication. Vous verrez dans le cours de Physique que la tension de la corde est donnée par la relation $T = \frac{mv^2}{r}$.

✎ Correction de l'exercice 6

La tension vaut $T = \frac{mv^2}{r}$: à masse m fixée, elle croît avec v. On teste donc les quatre vitesses possibles par ordre décroissant, à l'aide d'un enchaînement `if/elif`, en s'arrêtant à la première qui reste compatible avec $T \leqslant T_{max}$:

rupture.py

```
def rupture(m):
    r = 3
    T_max = 60
    if m*40**2/r <= T_max:
        return 40
    elif m*20**2/r <= T_max:
        return 20
    elif m*10**2/r <= T_max:
        return 10
    elif m*1**2/r <= T_max:
        return 1
    else:
        return 0
```

console

```
In [1]: rupture(0.05)
Out[1]: 40

In [2]: rupture(0.3)
Out[2]: 20

In [3]: rupture(1)
Out[3]: 10

In [4]: rupture(5)
Out[4]: 1

In [5]: rupture(200)
Out[5]: 0
```

Boucles `for` et `while`

7 Exercice

Donné un nombre réel x, sa partie entière notée $\lfloor x \rfloor$ est le plus grand entier relatif inférieur ou égal à x. Par exemple :

$$\lfloor -3 \rfloor = -3 \; ; \quad \lfloor 3{,}14 \rfloor = 3 \; ; \quad \lfloor -3{,}14 \rfloor = -4.$$

Sans utiliser le module `math`, écrire une fonction `floor` qui prend en paramètre un nombre x et qui renvoie l'entier $\lfloor x \rfloor$.

✎ Correction de l'exercice 7

On construit $\lfloor x \rfloor$ pas à pas avec une boucle `while`, en partant de 0 et en avançant (si $x \geqslant 0$) ou en reculant (si $x < 0$) d'une unité tant que ce n'est pas encore le bon entier :

floor.py

```
def floor(x):
    if x >= 0:
        n = 0
        while n + 1 <= x:
            n += 1
        return n
    else:
        n = 0
        while n > x:
            n -= 1
        return n
```

console

```
In [1]: floor(-3), floor(3.14), floor
   (-3.14)
Out[1]: (-3, 3, -4)
```

8 | EXERCICE

Soit la suite $(u_n)_n$ définie par $u_0 = 0$ et pour tout $n \in \mathbb{N}$,

$$u_{n+1} = u_n^2 + 1.$$

Écrire une fonction `u(n)` prenant en paramètre un entier positif n et qui retourne la valeur numérique du terme u_n de rang n de la suite $(u_n)_n$.

✎ CORRECTION DE L'EXERCICE 8

suiteu.py

```
def u(n):
    val = 0
    for k in range(n):
        val = val**2 + 1
    return val
```

9 | EXERCICE

Écrire une fonction `somme(n,k)` prenant en paramètres deux entiers positifs n et k et qui retourne la valeur de la somme $\sum_{a=1}^{n} a^k$.

Pour $k = 1, 2, 3$ comparer le résultat obtenu pour certaines valeurs de n que l'on choisira avec ce qu'on obtient par les formules connues :

- $\sum_{a=1}^{n} a = \frac{n(n+1)}{2}$
- $\sum_{a=1}^{n} a^2 = \frac{n(n+1)(2n+1)}{6}$
- $\sum_{a=1}^{n} a^3 = \frac{n^2(n+1)^2}{4}$

✎ CORRECTION DE L'EXERCICE 9

somme.py

```
def somme(n,k):
    s = 0
    for a in range(1,n+1):
        s += a**k
    return s
```

Pour $n = 10$ et $k = 1, 2, 3$, on retrouve bien les valeurs données par les formules connues :

console

```
In [1]: somme(10,1), 10*11/2
Out[1]: (55, 55.0)

In [2]: somme(10,2), 10*11*21/6
Out[2]: (385, 385.0)

In [3]: somme(10,3), 10**2*11**2/4
Out[3]: (3025, 3025.0)
```

10 | EXERCICE
Série harmonique

On admet que $\lim_{n\to\infty} \sum_{k=1}^{n} \frac{1}{k} = +\infty$.

Déterminer le plus petit entier n tel que $\sum_{k=1}^{n} \frac{1}{k} \geqslant 10$.

✎ CORRECTION DE L'EXERCICE 10

On accumule la somme terme après terme avec une boucle `while`, jusqu'à dépasser 10 :

harmonique.py

```
s = 0
n = 0
while s < 10:
    n += 1
    s += 1/n
print(n, s)
```

console

```
In [1]: (executing harmonique.py)
12367 10.000043008275778
```

Le plus petit entier n tel que $\sum_{k=1}^{n} \frac{1}{k} \geqslant 10$ est donc $n = 12\,367$ (la somme valant alors environ 10,000043).

11 | EXERCICE

Écrire une fonction `sommeDouble(n)` qui retourne la valeur numérique du résultat de la somme double :

$$\sum_{k=0}^{n} \sum_{i=0}^{k} (i + k).$$

Comparer pour certaines valeurs de n avec le résultat obtenu par le calcul.

✎ CORRECTION DE L'EXERCICE 11

On traduit directement la somme double par deux boucles `for` imbriquées :

sommeDouble.py

```python
def sommeDouble(n):
    s = 0
    for k in range(n+1):
        for i in range(k+1):
            s += i+k
    return s
```

En comparant pour quelques valeurs de n avec le calcul direct de la somme (on peut montrer que $\sum_{k=0}^{n}\sum_{i=0}^{k}(i+k) = \frac{n(n+1)(n+2)}{2}$) :

console

```
In [1]: sommeDouble(3), 3*4*5//2
Out[1]: (30, 30)

In [2]: sommeDouble(5), 5*6*7//2
Out[2]: (105, 105)

In [3]: sommeDouble(10), 10*11*12//2
Out[3]: (660, 660)
```

12 EXERCICE
Conjecture de Syracuse

Q1. Écrire une fonction `syracuse(u,n)` qui prend en paramètre deux entiers positifs `u` et `n` et qui retourne les $n+1$ premiers termes $u_0, u_1, \dots, u_n$ de la suite $(u_n)_n$ définie par $u_0 = u$ et pour tout $n \in \mathbb{N}$,

$$u_{n+1} = \begin{cases} \frac{u_n}{2} & \text{si } u_n \text{ est pair} \\ 3u_n + 1 & \text{si } u_n \text{ est impair} \end{cases}.$$

Q2. Vérifier sur quelques essais la conjecture de Syracuse : « Quel que soit son premier terme $u_0 \in \mathbb{N}^*$, la suite de Syracuse est périodique à partir d'un certain rang ».

Q3. Le plus petit indice n tel que $u_n = 1$ est appelé le **temps de vol** de la suite. Écrire une fonction `tempsdevol(u)` qui prend en argument le première terme u et retourne le temps de vol de la suite.

✎ CORRECTION DE L'EXERCICE 12

Q1.

syracuse.py

```python
def syracuse(u,n):
    terme = u
    print(terme)
    for k in range(n):
        if terme % 2 == 0:
            terme = terme // 2
        else:
            terme = 3*terme + 1
        print(terme)
```

console

```
In [1]: syracuse(6,10)
6
3
10
5
16
8
4
2
1
4
2
```

Q2. On observe bien, sur cet exemple ($u_0 = 6$) comme sur d'autres essais, que la suite atteint 1 puis boucle indéfiniment sur le cycle $1, 4, 2, 1, 4, 2, \dots$ ce qui illustre (sans la démontrer !) la conjecture de Syracuse.

Q3. Le temps de vol se calcule avec une boucle `while`, qui s'arrête dès que le terme courant vaut 1 :

tempsdevol.py

```python
def tempsdevol(u):
    n = 0
    terme = u
    while terme != 1:
        if terme % 2 == 0:
            terme = terme // 2
        else:
            terme = 3*terme + 1
        n += 1
    return n
```

console

```
In [1]: tempsdevol(1)
Out[1]: 0

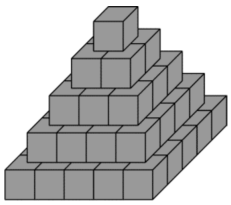
In [2]: tempsdevol(6)
Out[2]: 8

In [3]: tempsdevol(7)
Out[3]: 16

In [4]: tempsdevol(27)
Out[4]: 111
```

13 EXERCICE
Pyramide

On souhaite empiler des cubes pour former une pyramide : le premier étage (au sommet) est un carré d'un seul cube, le deuxième étage un carré de 4 cubes, le troisième un carré de 9 cubes, et ainsi de suite, chaque étage supplémentaire étant un carré de côté une unité de plus que le précédent.



On dispose d'un stock limité de cubes, et on souhaite savoir jusqu'à quelle hauteur on peut monter la pyramide sans l'épuiser.

Écrire une fonction `pyramide(n)` qui, étant donné le nombre `n` de cubes disponibles, renvoie un couple constitué de la hauteur maximale de pyramide qu'on peut construire sans dépasser ce stock, et du nombre de cubes alors effectivement utilisés.

console

```
In [1]: pyramide(37)
Out[1]: (4, 30)
```

✎ CORRECTION DE L'EXERCICE 13

On ajoute les étages un par un (de taille $1^2, 2^2, 3^2, \ldots$), tant que le stock `n` n'est pas dépassé :

pyramide.py

```
1 def pyramide(n):
2     hauteur = 0
3     total = 0
4     etage = 1
5     while total + etage**2 <= n:
6         total += etage**2
7         hauteur += 1
8         etage += 1
9     return hauteur, total
```

console

```
In [1]: pyramide(37)
Out[1]: (4, 30)
```